SystemVerilog中的类型转换

1.SystemVerilog中的类型转换

- 在SystemVerilog中，类型按照的兼容性分为5各层级，分别是匹配（matching）、相同类型（equivalent）、赋值兼容（assignment compatible）、转换兼容（cast compatible）和不相同类型（non-equivalent），其中匹配、相同类型其实也属于赋值兼容的一部分，所有的赋值兼容和所有的可以使用显式转换的不相同类型都属于可转换类型。
- 因此，可以知道，其实 **隐式转换主要存在于匹配和相同的数据类型之间，而显式转换存在于其他不同数据类型或者赋值兼容的数据类型之中。**而隐式数据类型转换，显式数据类型转换的主要区别是转换是否“肉眼可见”，隐式转换一般会“静默”完成，而显式需要用户使用特殊的语法结构才能实现

2.隐式转换

- 隐式转换一般情况是下表中的数据类型变量相互之间进行赋值时会自动完成类型转换

Table 6-8—Integer data types

`shortint`	2-state data type, 16-bit signed integer
`int`	2-state data type, 32-bit signed integer
`longint`	2-state data type, 64-bit signed integer
`byte`	2-state data type, 8-bit signed integer or ASCII character
`bit`	2-state data type, user-defined vector size, unsigned
`logic`	4-state data type, user-defined vector size, unsigned
`reg`	4-state data type, user-defined vector size, unsigned
`integer`	4-state data type, 32-bit signed integer
`time`	4-state data type, 64-bit unsigned integer

硅芯思见

```
1 module top_tb;
2 int          var1;
3 byte         var2;
4 bit    [3:0] var3;
5 logic [15:0] var4;
6 time         var5;
7 initial begin
8     var1 = 12345678;
9     var2 = 8'b1111_1111;
10    var3 = 4'b1010;
11    var4 = 16'h8765;
12    var5 = 8;
13    $display("var1 is %h",var1);
14    $display("var2 is %h",var2);
15    $display("var3 is %h",var3);
16    $display("var4 is %h",var4);
17    $display("var5 is %h",var5);
18    $display("==============");
19    var4 = 'hx;
20    var3 = var4;
21    $display("var3 is %h",var3);
22    $display("var4 is %h",var4);
23    $display("==============");
24    var2 = 8'b1000_0000;
25    var4 = var2;
26    $display("==============");
27    $display("var2 is %h",var2);
28    $display("var4 is %h",var4);
29 end
30 endmodule
```

硅芯思见

- 将var4赋值为不定态之后在赋值给var3，虽然var4具有四值数据类型，但是因为var3是二值数据类型，当var4赋值给var3时，其中的不定态自动转换为了0，这个转换过程是隐式自动完成的。
- var2是一个byte类型，为有符号数，示例中其最高位为1，因为var4位宽比var2宽，所以再将其赋值给var4时，需要对高位进行补位操作，这时的补位操作将根据var2最高位的特点进行补位，并且这个过程是隐式自动完成的。

2.显式转换

- 显示的类型转换主要有两种方式：
  - static conversion : type'(expr)
  - dynamic conversion : $cast(dest, src)
- **静态转换一般发生于编译阶段，在编译阶段完成类型的检查，而动态转换一般位于仿真运行阶段。**
- static静态转换:
  - 静态转换不会对转换值做检查，在编译阶如果发生转换失败则会报错，但是对于一般的“integer data types”之间在转换是产生的截位等问题则不会有相应的警告信息。
  - 企图将packet通过静态转换为int型，此时在代码编译阶段已经报错，即不支持将class对象转换为int型变量，其实在SystemVerilog中class对象不支持这种转换，基本上所有的不以连续位方式进行存储的变量在使用静态转换是都会报错。

```
1  class packet;
2      bit sig;
3  endclass
4
5  module top_tb;
6  packet pkt;
7  int    vart;
8
9  initial begin
10     vart = int'(pkt);
11 end
12 endmodule
```



```
Top Level Modules:
       top_tb
No TimeScale specified

Error-[SV-FNYI] Feature not yet implemented
top_tb.sv, 10
  SystemVerilog feature not yet implemented. Static casting involving class
  object Expression: pkt


1 error
CPU time: .238 seconds to compile

 common elaboration failed
make: *** [elab] Error 255
```

- 类型转换: casting_type’(expression)给int型变量赋值为十进制64，**然后通过”string’(vart)”的方式将整数值64转换成字符**，此时的仿真结果将按照ASCII码表中整数64对应的字符进行显示，显示的结果字符为”@”

```
`timescale 1 ns / 1 ps
module top_tb;
int vart;
initial begin
    vart = 64;
    $display("the cast result is %s",string'(vart));
end
endmodule
```

硅芯思见

```
the cast result is @
Simulation complete, time is 0 ps.
```

- 位宽转换: dst_size’(expression) 在SystemVerilog中可以将一个表达式结果的位宽转换为指定的位宽，这里需要注意不要转换的表达式位于括号中，要转换为的位宽必须为正数，不能为0或者负数。**示例中的src_vector为无符号数，无符号数在位宽扩展时高位补0，如果src_vector为有符号数，那么此时将src_vector转换成宽位宽的数时，此时需要注意该有符号数的符号位，此时的补位无论如何都将使用 0 进行补位。**

```
`timescale 1 ns / 1 ps
module top_tb;
bit [7:0] src_vector = 8'b1010_1010;
initial begin
    $display("the src_vector is %b",src_vector);
    $display("changed width src_vector is %b",10'(src_vector));
end
endmodule
```

硅芯思见

```
the src_vector is 10101010
changed width src_vector is 0010101010
Simulation complete, time is 0 ps.
```

- 符号转换: signed’(expression) 和unsigned’(expression)在SystemVerilog中还可以通过系统函数$signed和$unsigned实现一样的功能

```
`timescale 1 ns / 1 ps
module top_tb;
int       value0 = -8;
bit [3:0] value1 = 4'b1010;
initial begin
    $display("signed data in binary  is %b",value0);
    $display("signed data in decimal is %d",value0);
    $display("signed->unsigned data in binary  is %b",unsigned'(value0));
    $display("signed->unsigned data in decimal is %d",unsigned'(value0));
    $display("==============================");
    $display("unsigned data is %d",value1);
    $display("unsigned->signed data is %d",signed'(value1));
end
endmodule
```

硅芯思见

```
signed data in binary  is 11111111111111111111111111111000
signed data in decimal is           -8
signed->unsigned data in binary  is 11111111111111111111111111111000
signed->unsigned data in decimal is 4294967288
==============================
unsigned data is 10
unsigned->signed data is           -6
Simulation complete, time is 0 ps.
```

硅芯思见

- dynamic动态转换:动态转换的过程相较于静态转换主要区别在于该转换和检查发生在仿真运行阶段
  - function int $cast(singular dest_var,singular src_exp)
  - task $cast(singular dest_var,singular src_exp)
  - $cast作为任务task
    - 虽然子类**scobj**是从父类**cobj**拓展派生而来的，两个属于不同的类型，但是父类句柄是可以直接指向子类对象，子类句柄是不能直接指向父类对象的

```
`timescale 1 ns / 1 ps
module top_tb;
class cobj;
    int i = 1;
endclass

class scobj extends cobj;
    int j = 2;
endclass

cobj  p;
scobj sp;

initial begin
    p = new();
    $cast(sp,p);
end
endmodule
```

硅芯思见

```
Error-[DCF] Dynamic cast failed
top_tb.sv, 16
  Casting of source class type 'cobj' to destination class type 'scobj' failed
  due to type mismatch.
  Please ensure matching types for dynamic cast

Simulation complete, time is 0 ps.
```

硅芯思见

- $cast作为函数function
  - 仿真并没有因为这次不匹配报出任何提示信息和停止仿真

```
`timescale 1 ns / 1 ps
module top_tb;
class cobj;
    int i = 1;
endclass

class scobj extends cobj;
    int j = 2;
endclass

cobj  p;
scobj sp;
int   tmp;

initial begin
    p   = new();
    tmp = $cast(sp,p);
    if(tmp) begin
        $display("1st : $cast successfully,the cast func return is %0d",tmp);
    end else begin
        $display("1st : $cast failed,the cast func return is %0d",tmp);
    end
    tmp = $cast(p,sp);
    if(tmp) begin
        $display("2nd : $cast successfully,the cast func return is %0d",tmp);
    end else begin
        $display("2nd : $cast failed,the cast func return is %0d",tmp);
    end
end
endmodule
```

硅芯思见

```
1st : $cast failed,the cast func return is 0
2nd : $cast successfully,the cast func return is 1
Simulation complete, time is 0 ps.
```

硅芯思见

3.总结

- 当将$cast作为任务使用时，如果源对象的数据类型与目的对象的数据类型不匹配不兼容仿真时会报错，并且仿真停止；
- **当将$cast作为函数使用时，如果源对象的数据类型与目的对象的数据类型不匹配不兼容仿真时，该函数仅返回值0，并且仿真不会停止也不会给出任何提示，如果类型兼容匹配，转换成功，那么函数返回非零值；**
- $cast具体作为函数还是任务，取决于用户在编码时按照如何的方式对$cast进行使用，如果期望类型不匹配不兼容时能够给出信息并且停止仿真，那么用户应该像任务一样使用$cast，如果想根据类型转换的结果进行特定的操作并不自动停止时，那么用户应该像函数一样使用$cast。